BRACE

Architectural Comparison

Subsumption Architecture · ROS · BRACE

1. Introduction

Every robot control system must answer the same fundamental question: how should competing demands on hardware — sensing, movement, safety, mission logic — be organized and resolved? Different architectural philosophies give very different answers, with important consequences for how easy the system is to build, debug, extend, and scale.

This document compares the BRACE used in this course against two influential reference architectures: Rodney Brooks's Subsumption Architecture (1986) and the Robot Operating System (ROS). The comparison examines how each architecture handles behavior organization, hardware arbitration, communication, sensing, and extensibility — and identifies where the BRACE deliberately echoes these designs and where it diverges.

2. Architecture Summaries

2.1 Subsumption Architecture

Proposed by Rodney Brooks at MIT, the Subsumption Architecture organizes robot behavior as a fixed hierarchy of independent layers. Each layer is a finite-state machine running concurrently, directly connected to sensors and actuators with no shared world model. Higher layers can suppress the outputs of lower layers or inhibit their inputs, but they cannot communicate with them in any other way.

The key insight is that intelligent behavior can emerge from the interaction of simple reactive rules, without any explicit planning or centralized representation of the world. A robot navigating around obstacles does not need to know where it is — it only needs a low-level layer that says "if something is close, turn away."

Property	Subsumption Architecture
Behavior organization	Fixed priority hierarchy of concurrent FSM layers
Hardware access	Direct — each layer reads sensors and drives actuators independently
Inter-layer communication	Suppression and inhibition signals only — no shared data
World model	None — purely reactive to current sensor readings
Concurrency model	All layers run in parallel at all times
Typical use	Simple mobile robots, insect-like behaviors, real-time embedded systems

2.2 ROS (Robot Operating System)

ROS is a middleware framework developed at Stanford and Willow Garage, now maintained by Open Robotics. It provides a publish/subscribe communication graph where independent processes (nodes) exchange typed messages over named channels (topics). Nodes can also expose request/response interfaces (services) and long-running goal-oriented interfaces (actions).

ROS makes no assumptions about robot hardware, behavior organization, or programming language. It is a coordination infrastructure: it handles discovery, serialization, logging, visualization, simulation, and parameter management, allowing engineers to compose complex systems from reusable, independently developed nodes.

Property	ROS
Behavior organization	Arbitrary graph of nodes; behavior trees or state machines typically layered on top
Hardware access	Device driver nodes publish sensor data and subscribe to actuator commands
Inter-node communication	Publish/subscribe topics, services (request/response), actions (goal/feedback/result)
World model	Shared via topics; tf2 maintains a live coordinate frame tree
Concurrency model	Each node is a separate process; intra-node threading optional
Typical use	Research robots, autonomous vehicles, industrial arms, multi-sensor fusion systems

2.3 BRACE

The BRACE is a two-tier client/server system. The Python server running on the CyberPi manages hardware directly, schedules concurrent behavior loops, and arbitrates access to shared resources. The Java host provides a high-level command API and a sensor cache updated by a UDP telemetry stream, allowing a state-machine controller (MazeRobot) to make decisions without directly touching hardware.

Property	BRACE
Behavior organization	Named behavior functions in a priority-based scheduler; state machine in Java controller
Hardware access	Python ResourceArbiter mediates all hardware access by priority
Communication	TCP command/response + UDP telemetry between Java host and Python server
World model	SensorCache on the Java side — a lightweight volatile-field snapshot updated at ~100 Hz
Concurrency model	Python _thread per behavior; Java TelemetryListener thread + main controller thread
Typical use	Educational robotics; structured enough for a course project, simple enough to be understood in full

3. Detailed Comparison

3.1 Behavior Organization and Priority

How competing behaviors are organized and which one "wins" when they conflict is the central design decision in any robot architecture.

Subsumption

Layers are numbered and their priority is structural — layer N always overrides layer N−1 by suppressing its actuator output or inhibiting its sensor input. There is no dynamic arbitration: the hierarchy is fixed at design time and enforced by wiring, not by runtime logic. This makes the priority model completely transparent and formally verifiable, but it also means the hierarchy cannot adapt to context.

ROS

ROS itself imposes no behavior hierarchy. Priority and conflict resolution are delegated entirely to the application. Common approaches include behavior trees (with priority-ordered fallback nodes), state machines (SMACH, FlexBE), or deliberative planners (MoveIt, nav2). The result is highly flexible but requires the developer to consciously design the arbitration logic — it is not provided by the framework.

BRACE

The BRACE uses a **numeric priority arbiter** that is conceptually similar to subsumption but applied dynamically at runtime. Each behavior and command handler declares a priority integer when it acquires a hardware resource. A higher-priority acquire evicts the current owner using the `>=` rule. This gives the system a runtime-configurable priority hierarchy rather than a fixed wired one.

The key difference from pure subsumption is that the BRACE arbiter operates on named resources (motors, ultrasonic, line, camera) rather than on actuator output lines. A behavior does not suppress another behavior's output directly — it evicts its hardware lock. This is a finer-grained model that allows, for example, a safety behavior to take over the motors without affecting sensor reads happening simultaneously in other behaviors.

> **Design echo:** The BRACE priority arbiter directly reflects the subsumption insight that safety-critical behaviors must structurally guarantee they can always act. AVOID_CRASHING at priority 250 can never be blocked by any other running behavior — this is the same guarantee that a subsumption layer provides through suppression wiring.

3.2 Communication Model

How components exchange information shapes how modular, debuggable, and network-transparent a system is.

Subsumption

Communication is local and hardware-level: wires carry suppression/inhibition signals between layers. There is no message passing, no serialization, and no network. This makes the system extremely fast and deterministic but also tightly coupled to a specific hardware configuration.

ROS

ROS uses a publish/subscribe model over a named topic graph. Publishers and subscribers are decoupled — a sensor node publishes on /scan without knowing who reads it. This enables powerful patterns: multiple nodes can consume the same sensor data, nodes can be replaced or mocked without changing their consumers, and the full message stream can be recorded (rosbag) and replayed for debugging. The cost is serialization overhead and non-deterministic delivery timing on a shared bus.

BRACE

The BRACE uses two distinct channels. The **TCP command channel** is a synchronous request/response protocol: the Java host sends a JSON command and blocks until the robot replies. The **UDP telemetry channel** is an asynchronous broadcast that loosely mirrors a ROS topic: the robot publishes sensor data continuously, and any registered client consumes it independently of the command flow.

This hybrid is a pragmatic simplification of the ROS model. Commands benefit from synchronous confirmation (the Java host knows when a motor command completed), while high-frequency sensor data benefits from fire-and-forget UDP delivery that does not stall the command pipeline.

> **Design echo:** The BRACE TelemetryStreamer / SensorCache pair directly mirrors the ROS publisher/subscriber pattern. The Python server publishes sensor state; the Java SensorCache subscribes and maintains a local snapshot. The awaitNewData() method in RobotController plays the role of a ROS subscriber callback — it yields until fresh data is available.

3.3 Sensing and World Model

How sensor data is collected, shared, and made available to decision-making logic differs significantly across the three architectures.

Subsumption

There is no world model. Each layer reads sensors directly and reacts immediately. Sensor data is not stored, aggregated, or shared between layers. This avoids the complexity of maintaining consistent shared state but limits the system to purely reactive behaviors — planning ahead is impossible because there is no representation of past observations.

ROS

ROS supports rich world modelling through topic composition. A LIDAR node publishes raw scans; a mapping node subscribes and builds an occupancy grid; a localization node fuses the map with odometry to estimate position; a planning node uses the position estimate to generate a path. Each node maintains its own local state, and the global picture emerges from the composition of these local views. The tf2 library maintains a live tree of coordinate frame transformations, allowing any node to ask "where is frame X relative to frame Y right now?"

BRACE

BRACE `SensorCache` is a minimal world model: a set of volatile fields holding the most recent reading from each sensor. It is updated at ~100 Hz by the `TelemetryListener` and read by the Java state machine without locking (volatile guarantees visibility). There is no history, no map, no coordinate frame tree, and no sensor fusion — only the current snapshot.

This is a deliberate simplification. For a maze-solving task on a small arena, the current distance and line status are sufficient for decision-making. The SensorCache would need to be substantially extended — adding history, timestamps, or a spatial model — for tasks requiring localization or path planning.

> **Design contrast:** This is the sharpest difference from ROS. A ROS-based equivalent would use a costmap, a tf2 transform tree, and an AMCL localization node. The BRACE SensorCache

trades all of that richness for simplicity and low resource consumption — appropriate for a microcontroller-class platform running MicroPython.

3.4 Concurrency and Hardware Access

Managing concurrent access to shared hardware (motors, sensors) without conflicts or race conditions is a core challenge in robot software.

Subsumption

Concurrency conflicts are avoided structurally. Because suppression wires guarantee that only one layer's output reaches the actuator at any time, there is no shared resource to protect. The hardware arbitration is built into the architecture's topology.

ROS

ROS nodes are separate processes, so there are no shared memory races between nodes. Hardware drivers serialize access within a single node using callbacks and a single-threaded executor by default. When multiple nodes must share a resource (e.g., two nodes both wanting to publish velocity commands), a multiplexer node (twist_mux) is typically added to the graph to implement priority-based selection — effectively inserting a software arbiter into the communication graph.

BRACE

The `ResourceArbiter` is the BRACE equivalent of both the subsumption suppression wire and the ROS twist_mux. It is a single lock-protected dictionary mapping resource names to (owner, priority) pairs. Any thread that wants to use a hardware resource must acquire it first. A higher-priority acquire immediately evicts the current owner by replacing the dictionary entry.

This design is simpler than either reference architecture but more explicit than subsumption (the priority is a runtime number, not a wired topology) and more centralized than ROS (one arbiter governs all resources, rather than per-node executors and per-resource multiplexers). The trade-off is that all behaviors must cooperate with the arbiter protocol — a behavior that bypasses the arbiter can corrupt motor state without any system-level protection.

3.5 Extensibility

How easy is it to add a new behavior or capability to each system?

Subsumption

Adding a new behavior means adding a new layer with suppression/inhibition connections to the appropriate points in the existing hierarchy. In hardware implementations this means physical

rewiring; in software it requires modifying the FSM layer topology. The architecture does not scale well beyond five or six layers — the suppression graph becomes difficult to reason about, and the interaction between many layers can produce emergent behaviors that are hard to predict or debug.

ROS

ROS is designed for extensibility. A new capability is a new node that publishes and subscribes to topics. It can be developed, tested, and deployed independently of other nodes. The graph can be inspected live with tools like rqt_graph, and individual topics can be monitored with ros2 topic echo. The cost of this flexibility is the overhead of learning the ROS ecosystem — launch files, package manifests, message type definitions, and the colcon build system.

BRACE

The BRACE uses a `@register_command` decorator to add new commands with zero boilerplate, and the `scheduler.start_behavior()` API to add new background behaviors. Both additions are confined to the Extension Zone at the bottom of a single file, making the contribution surface small and easy to locate.

On the Java side, a new BRACE method requires only a call to execute() with the matching command name — no registration, no interface changes, no build configuration. This is the most accessible extension model of the three, at the cost of weaker isolation: a badly written behavior can affect the whole server because all behaviors share a single Python process and the global arbiter.

4. Side-by-Side Comparison

Aspect	Subsumption	ROS	BRACE	Key Distinction
Behavior priority	Fixed structural hierarchy — wired suppression	No built-in priority — delegated to app layer	Dynamic numeric priority per resource acquire	BRACE combines subsumption's guarantee (safety always wins) with runtime configurability
Communication	Suppression / inhibition signals only	Pub/sub topics, services, actions over DDS	TCP request/response + UDP telemetry broadcast	BRACE UDP telemetry mirrors a ROS topic; TCP commands mirror a ROS service
World model	None — purely reactive	Rich — costmaps, tf2 frames, sensor fusion	Minimal — SensorCache volatile snapshot at ~100 Hz	BRACE is reactive like subsumption but stores the latest reading; no history or map

Hardware access	Direct per-layer — no shared state to protect	Per-node executors; twist_mux for motor arbitration	Centralized ResourceArbiter with owner + priority	BRACE arbiter is explicit where subsumption relies on topology and ROS on node isolation
Sensor integration	Each layer reads its own sensors directly	Sensor driver nodes publish on topics	Python reads hardware; Java consumes via UDP	BRACE crosses a network boundary that subsumption and single-machine ROS do not
Concurrency model	All layers run in parallel — no shared memory	Separate OS processes per node	Shared-memory Python threads + arbiter lock	BRACE requires explicit cooperative locking; the others have structural or process isolation
Adding behaviors	New FSM layer + suppression wiring	New node + topic subscriptions	@register_command decorator + scheduler call	BRACE has the lowest barrier to entry; ROS has the best isolation
Debugging / observability	Very limited — no message logging	Rich — rqt, rosbag, topic echo, node graph viz	Console print on CyberPi + Java stdout	ROS has by far the strongest tooling; BRACE and subsumption rely on manual logging
Platform dependency	Minimal — runs on bare metal	Linux required; ROS 2 supports limited RTOS	CyberPi (MicroPython) + JVM host	BRACE spans two platforms and two languages; subsumption needs neither OS nor network
Scalability	Breaks down beyond ~6 layers	Scales to hundreds of nodes	Scales to tens of behaviors on one CyberPi	ROS is the only architecture designed for large-scale composition

5. Architectural Lineage of the BRACE

The BRACE is not designed from first principles — it deliberately borrows recognizable patterns from both reference architectures, adapted to the constraints of an educational platform.

5.1 What Comes from Subsumption

- **Priority-based hardware arbitration.** The ResourceArbiter's numeric priority is a software rendering of subsumption's fixed suppression hierarchy. AVOID_CRASHING at priority 250 is the direct analogue of Brooks's obstacle-avoidance layer that always overrides the wander layer.
- **Reactive sensing without a world model.** Like subsumption layers, BRACE behaviors read current sensor values and react immediately. The SensorCache holds only the latest reading — there is no temporal integration, no map, no localization.

- **Concurrent behavior loops.** The BehaviorScheduler runs each behavior as an independent looping thread, mirroring subsumption's parallel FSM layers. Each behavior is designed to be self-contained and oblivious to what other behaviors are doing.

5.2 What Comes from ROS

- **Publish/subscribe telemetry.** The TelemetryStreamer / SensorCache pair is structurally a ROS publisher/subscriber pair. The Python server publishes sensor state over UDP; the Java client subscribes by receiving packets and updating the cache. The awaitNewData() pattern is equivalent to blocking on a ROS subscriber callback.
- **Named command registry.** The @register_command decorator pattern echoes the ROS service server model: a named endpoint is declared, and any caller who knows the name can invoke it. The handler is decoupled from the transport, just as a ROS service callback is decoupled from the DDS layer.
- **Separation of sensing and control.** ROS separates hardware driver nodes from decision-making nodes. The BRACE does the same across a network boundary: the Python server is the driver layer that owns hardware; the Java host is the decision-making layer that owns the state machine. Neither layer directly accesses the other's domain.

5.3 What the BRACE Does Not Have

- **Coordinate frames and localization (ROS tf2).** The robot has no map, no odometry integration, and no notion of where it is in the arena. All navigation is reactive.
- **Message type safety (ROS interfaces).** Commands and responses are untyped JSON dictionaries. A misspelled parameter key produces a silent default, not a compile-time or schema error.
- **Process isolation (ROS nodes).** All Python behaviors share one process. A behavior that crashes or enters an infinite loop can affect the entire server.
- **Emergent behavior composition (subsumption topology).** Subsumption's suppression wiring allows complex behaviors to emerge from simple layer interactions without explicit if/else logic. The BRACE state machine in MazeRobot requires explicit case logic for every state transition.
- **Formal verifiability.** Subsumption architectures can be formally analyzed because the suppression graph is a static, finite structure. The BRACE arbiter's runtime priority assignments are harder to reason about statically.

6. Evolutionary Path: From BRACE to Production Architectures

Understanding the BRACE as a simplified hybrid of subsumption and ROS makes it a useful pedagogical steppingstone. The following changes would move it toward each reference architecture.

Toward Subsumption

- Replace the numeric priority arbiter with a fixed suppression graph — each behavior has a hardwired set of behaviors it can suppress, enforced at compile time rather than runtime.
- Remove the network layer and move all control logic onto the CyberPi itself, eliminating the TCP round-trip latency from the sense–decide–act loop.
- Remove the SensorCache and have each behavior read sensors directly on every iteration.

Toward ROS

- Replace the custom TCP/UDP protocol with DDS (the ROS 2 transport), giving message type safety, automatic discovery, and QoS policies.
- Split each behavior into its own ROS node, gaining process isolation and the ability to restart a single behavior without affecting others.
- Add a costmap node and a tf2 broadcaster to give the robot a spatial model of its environment, enabling path planning and localization.
- Replace the JSON command registry with typed ROS service and action interfaces, catching parameter errors at the interface definition level rather than at runtime.